

Current Blockers

January 14, 2026

1 Drylab VM

Drylab VM is required for development / testing before deploying at the beamline. Currently all development is being done locally on a laptop and VM based implementation is only being done at the beamline during maintenance/shutdown. A working Drylab VM will allow a robust implementation with less time spent debugging issues when at the beamline.

1. VM is @ 10.69.26.42 / lob4-ros1
2. Robot is @ 10.69.26.41

1.1 Socket Communication issue

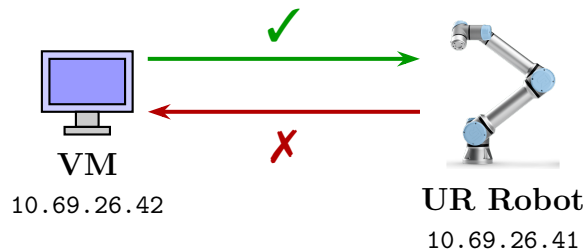


Figure 1: VM can connect to robot, but robot cannot communicate back to VM.

1. Robot successfully connects with the VM and connection is verified by ping
2. UR Cap (UR Software plugin to allow controlling the robot from PC/laptop) requires ports 50001-50004 for communication.(currently fails)
3. Verified the socket communication issue by running a test socket server / client between laptop and VM (no robot)
 - (a) Laptop as server , VM as client - works ✓
 - (b) Laptop as client and VM as server - fails ✗
4. Possible solution is to modify */etc/nftables/main.nft* to open required ports in the VM.
5. Communication works without issues at beamline VM.

1.2 Cameras

1. Zivid Camera requires an IP at drylab - preferably on the same subnet as the robot.
2. ZED cameras require setting up the Digi USB anywhere (USB → Ethernet) as well as IP.
3. Require GPU resource

2 Beamline VM

2.1 Cameras

1. Zivid Camera is not discoverable from VM (Camera is on CAM subnet , VM is on INST subnet?)
 - (a) either reconfigure the VM to have additional interface with CAM subnet
 - (b) or configure the camera in the same subnet as the robot (as an extension of the robot)
2. ZED cameras require Digi Anywhere USB to be setup.
3. All cameras require GPU compute which is currently unavailable.

2.2 Bluesky/EPICS

1. Bluesky on VM cannot communicate with EPICS/other beamline hardware.

bsui Container Launch Command:

```
#!/bin/bash

podman run -it --network host --ipc=host --pid=host \
  --env BS_ENV=$(basename "$BS_ENV") \
  --env BS_PROFILE="collection" \
  --env CDC_LOCALHOST=xf28id1-srv2.nsls2.bnl.local \
  --env EPICS_CA_ADDR_LIST=10.66.219.255 \
  --env EPICS_CA_AUTO_ADDR_LIST=no \
  --mount type=bind,source=/nsls2/conda/envs,target=/opt/conda/envs,
  readonly \
  --mount type=bind,source=/nsls2/data/pdf/shared/config/bluesky/
  profile_collection,target=/root/.ipython/profile_collection,readonly \
  --mount type=bind,source=/nsls2/software/etc/tiled/profiles,target=/
  root/.config/tiled/profiles,readonly \
  --mount type=bind,source=/etc/bluesky,target=/etc/bluesky,readonly \
  ghcr.io/nsls2/erobs-bsui:latest
```

3 GPU Requirements

Both Zivid and ZED camera systems require dedicated NVIDIA GPU resources for point cloud processing. This is a hard requirement; the cameras cannot function without GPU acceleration.

Requirement	Zivid 2+	ZED 2i	Current Setup (1× Zivid + 2× ZED 2i)
Min. GPU Memory (VRAM)	3 GB	6+ GB	12+ GB
Min. GPU	GTX 1060+	RTX 20-series+	RTX 40-series+
Interface	10GbE	USB 3.0	10GbE + 2× USB 3.0
Min. RAM	16 GB	8 GB	32 GB
OS Support	Ubuntu 22.04+	Ubuntu 22.04+	Ubuntu 22.04

Table 1: GPU and system requirements comparison: Zivid vs ZED 2i cameras.

Note: Data center GPUs (NVIDIA H100, A100, etc.) are not officially supported by either Zivid or ZED SDKs.

References:

- Zivid: <https://support.zivid.com/en/latest/getting-started/software-installation/gpu/gpu-requirements.html>
- ZED 2i: <https://www.stereolabs.com/docs/development/zed-sdk/specifications>

3.1 Zivid Capture Time Benchmarks

The figure below shows capture times for different GPU configurations with Manufacturing presets on Zivid 2+R.

	2+R					
	High-end NVIDIA		Low-end Intel		Low-end Jetson NVIDIA	
	2D	3D	2D	3D	2D	3D
Manufacturing Settings						
Manufacturing Diffuse	36 ms	150 ms 180 ms	125 ms	565 ms 590 ms	270 ms	520 ms 545 ms
Manufacturing Semi-Specular	36 ms	730 ms 760 ms	145 ms	1530 ms 1550 ms	270 ms	1160 ms 1190 ms
Manufacturing Specular	55 ms	955 ms 1000 ms	140 ms	1530 ms 1570 ms	290 ms	1265 ms 1305 ms
Manufacturing Small Features	49 ms	865 ms 900 ms	375 ms	4765 ms 4785 ms	1025 ms	3575 ms 3600 ms

Figure 2: Zivid 2+R capture times (ms) for Manufacturing presets across GPU configurations.

4 Deployment Comparison

Feature	PDF VM (Phil)	CMS VM (Current)	VM + GPU	Ubuntu Workstation	Laptop
Bluesky/EPICS Access	✓	✗	✓	✓	✗
GPU Available	✗	✗	✓	✓	✓
Camera Integration	physical server	✗	—	✓	✓

Table 2: Comparison of deployment options for EROBS system.

Legend: ✓ = Available/Working ✗ = Not Available/Failing ~ = Partial/Limited
 — = To Be Determined

5 EROBS Deployment Architecture

5.1 PDF Beamline implmentation by Phil

1. Two Docker containers - one for Bluesky (bsui) and one for ROS2 (erobs)
2. Only supports pick and place actions with predefined joint angles
3. Camera - Azure Kinect DK connected via USB to a physical server (System 76 Meerkat)

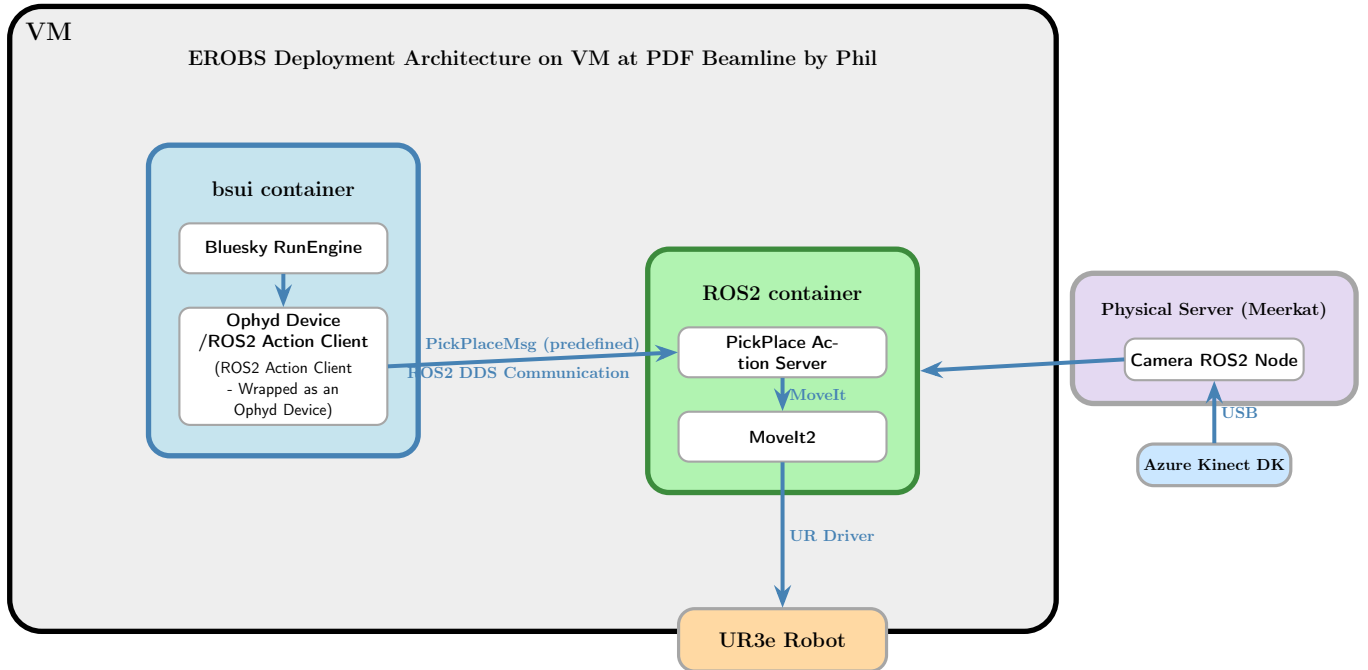


Figure 3: EROBS deployment architecture showing the two-container setup on VM at PDF beamline.

5.2 CMS Beamline Implementation on a VM

1. Two Docker containers - one for Bluesky (bsui) and one for ROS2 (erobs-common-img)
2. Full MTC orchestrator with multiple action servers (move_to, pick_place, vision, etc.)
3. JSON-based task interface for flexible experiment orchestration

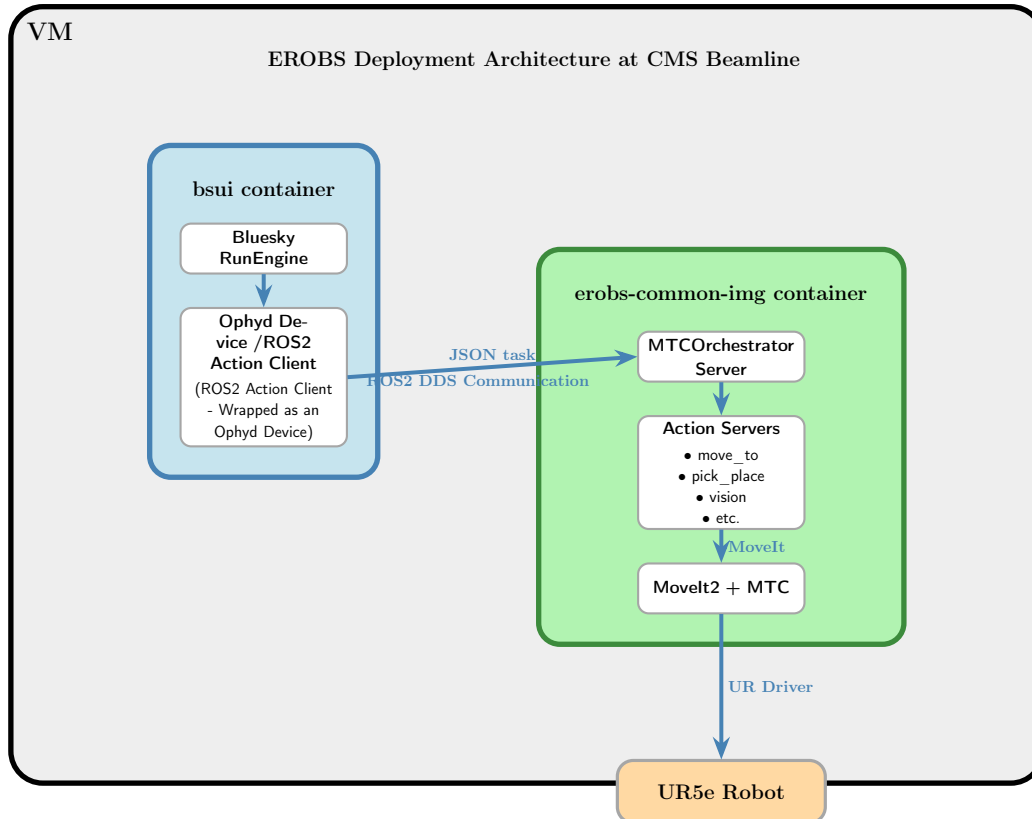


Figure 4: EROBS deployment architecture showing the two-container setup on VM at CMS beamline.

5.3 Ubuntu Workstation Implementation

1. Native installation - no Docker containers, all components run directly on the workstation
2. Direct camera integration via Camera Node (Zivid 10GbE, ZED USB)
3. GPU available for camera point cloud processing

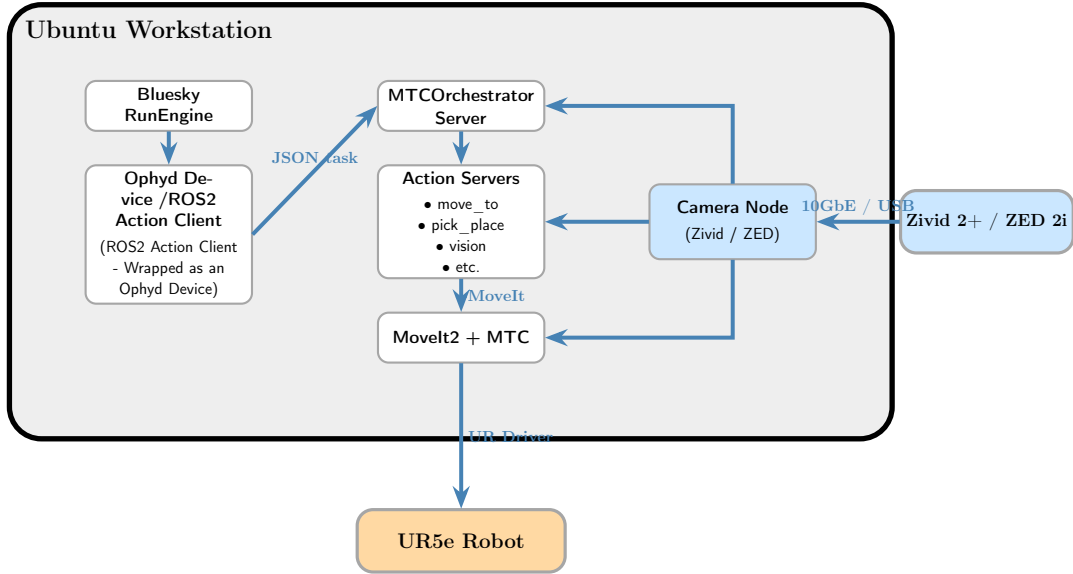


Figure 5: EROBS deployment architecture on Ubuntu Workstation.

5.4 Laptop Implementation

1. Native installation - no Docker containers, all components run directly on the laptop
2. Direct camera integration via Camera Node (Zivid 10GbE, ZED USB)
3. GPU available for camera point cloud processing

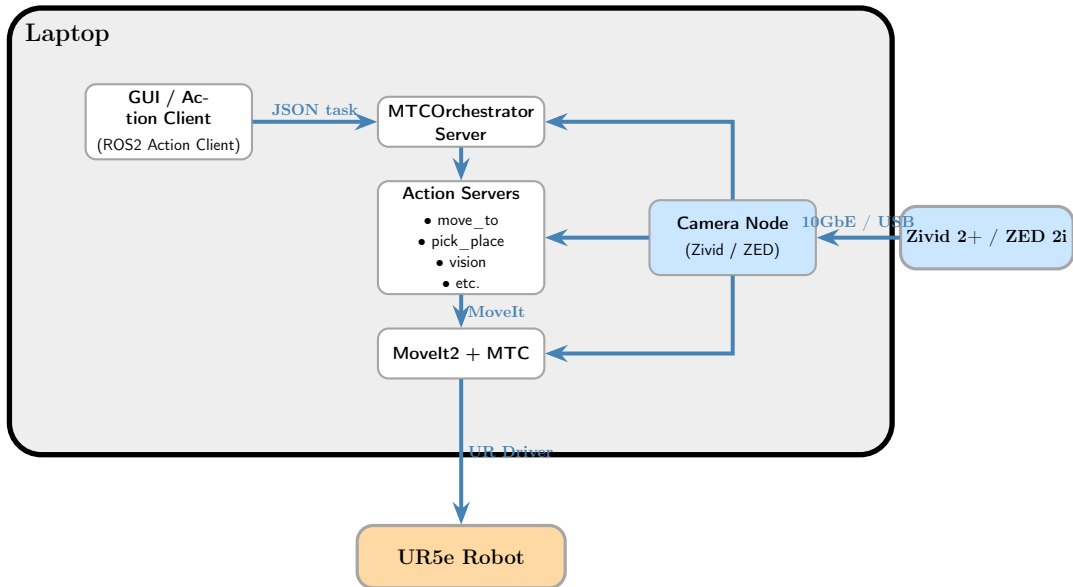


Figure 6: EROBS deployment architecture on Laptop.